Playwright + Cucumber Automation Framework Design Conversation

This document contains the structured conversation and implementation plan for designing an enterprise-grade Playwright Automation Framework using Cucumber, TypeScript, Parallel Execution, JSON Test Data Management, Cucumber HTML Reporting, and GitHub Actions CI/CD integration.

Selected Configuration

• Language: TypeScript
• Execution: Parallel Execution
• Reporting: Cucumber HTML Reports
• Environment: Single Environment
• Design Pattern: Page Object Model (POM)
• Test Data: JSON-based
• CI/CD: GitHub Actions

Framework Structure

playwright-cucumber-framework/
■■■ .github/workflows/playwright.yml
■■■ config/
■■■ src/features/
■■■ src/pages/
■■■ src/utils/
■■■ src/test-data/
■■■ reports/
■■■ cucumber.js
■■■ tsconfig.json
■■■ package.json

Implementation Steps

1. Project Initialization & Dependency Setup
2. TypeScript Configuration
3. Playwright Configuration
4. Cucumber Configuration
5. Core Framework Utilities
6. Page Object Model Implementation
7. BDD Feature & Step Definitions
8. JSON Test Data Integration
9. Reporting Integration
10. GitHub Actions CI/CD Integration

Step 1 — Dependency Setup

Commands used:

```
mkdir playwright-cucumber-framework
cd playwright-cucumber-framework
npm init -y

npm install -D @playwright/test playwright typescript ts-node @types/node
npm install -D @cucumber/cucumber multiple-cucumber-html-reporter cross-env rimraf
```

Step 2 — TypeScript Configuration

Configured:
• ES2020 target
• CommonJS modules
• Path aliases
• Strict mode
• JSON module support

Step 3 — Playwright Configuration

Configured:
• Parallel execution
• Retry strategy
• Screenshots on failure
• Video & trace retention
• Headless/Headed support

Step 4 — Cucumber Configuration

Configured:
• cucumber.js
• Hooks
• JSON reports
• HTML reports
• Parallel execution support

Step 5 — Framework Utilities

Utilities implemented:
• Browser Manager
• Wait Utility
• Screenshot Utility
• Logger Utility
• BasePage abstraction

Step 6 — POM Implementation

Implemented:
• LoginPage.ts
• Encapsulated locators
• Reusable actions
• Assertion helpers

Step 7 — BDD Implementation

Implemented:
• Feature files
• Step definitions
• Hooks integration
• Scenario outlines
• Parallel-safe execution

Step 8 — JSON Test Data Integration

Implemented:
• loginData.json
• DataUtil.ts
• Dynamic data loading
• Data-driven scenarios

Step 9 — Reporting Integration

Implemented:
• multiple-cucumber-html-reporter
• Screenshot attachments
• HTML execution reports
• Automated cleanup scripts

Step 10 — GitHub Actions CI/CD

Implemented:
• GitHub Actions workflow
• Node.js setup
• Dependency caching
• Artifact uploads
• Parallel test execution

Final Framework Capabilities

■ Playwright Automation
■ Cucumber BDD
■ TypeScript Support
■ Parallel Execution
■ Page Object Model
■ JSON Test Data
■ Screenshot Capture
■ HTML Reporting
■ GitHub Actions CI/CD

This PDF consolidates the complete automation framework design discussion and implementation guidance shared throughout the conversation.